

Phase 2: Project Structure Generation

1. System Analysis

System Type: Web App

Features:

- User Registration & Login
- Book Search
- Borrowing & Returning Books
- Email Reminders
- Admin Management
- Real-Time Availability Tracking
- Report Generation
- User Session Handling

Non-Functional Requirements:

- Encrypted password storage (REQ-010)
- Compatibility with modern web browsers (REQ-011)

Complexity: Medium

2. Architecture

Style: Layered Architecture

Pattern: Model-View-Controller (MVC)

Justification:

The chosen system analysis indicates a medium complexity web application, which is not suitable for microservices. Layered architecture is appropriate as it provides a well-defined structure with clear separation between the presentation layer, business logic, and data access layer, making it easier to manage and maintain. For the design pattern, the Model-View-Controller (MVC) is chosen due to its ability to handle the multiple features like User Registration & Login, Book Search, Borrowing & Returning Books, Email Reminders, Admin Management, Real-Time Availability Tracking, Report Generation, and User Session Handling efficiently while promoting loose coupling between components.

3. Tech Stack

Frontend: React.js or Angular.js

Backend: Node.js with Express.js

Database: MongoDB or PostgreSQL

Tools:

- Webpack
- Babel

Justification:

For the frontend, React.js or Angular.js are suitable as they are popular JavaScript libraries for building user interfaces and offer a rich set of components to efficiently handle multiple features like User Registration & Login, Book Search, Borrowing & Returning Books, Email Reminders, Admin Management, Real-Time Availability Tracking, Report Generation, and User Session Handling.

Node.js with Express.js is chosen for the backend due to its non-blocking nature, making it suitable for handling concurrent requests which are crucial for the Borrowing & Returning Books, Real-Time Availability Tracking features. Additionally, it offers a robust set of features for building APIs to facilitate communication between the frontend and backend. MongoDB or PostgreSQL can be used as the database depending on the need for a NoSQL or SQL solution respectively. Webpack and Babel are chosen for their ability to simplify the process of managing assets and transpiling code in a JavaScript project.

4. Project Structure

```
...
/my-library-app
  frontend
    src
      components
        Auth
        Books
        Admin
        Realtime
        Reports
        Sessions
      services
      utils
      assets
      App.js
    package.json
    ... (other essential frontend files)
  backend
    src
      controllers
        AuthController.js
        BookController.js
        AdminController.js
        RealtimeController.js
        ReportController.js
      models
        User.js
        Book.js
        ... (other models)
      routes
        auth.js
        books.js
        admin.js
        realtime.js
        reports.js
      config
        db.js
        ... (other configurations)
      app.js
    package.json
    ... (other essential backend files)
  database
    ... (database connection details)
  .gitignore
  README.md
```

